caugi: fast and flexible causal graph interface for R

Frederik Fabricius Bjerre[12], Johan Larsson[2], **Bjarke Hautop Kristensen**[1†], Michael C Sachs[1]

[1] Section of Biostatistics, University of Copenhagen,

[2] Department of Mathematical Sciences, University of Copenhagen,

[†] bjarke.hautop@gmail.com

R package, causal inference

Motivation

- Existing packages use low-level representations (e.g., adjacency matrices) and are not specialized for causal graphs.
- Causal workflows often jump between graph objects, adjacency matrices, and package-specific APIs.
- **Goal:** provide a graph interface that is expressive, safe, and efficient for causal inference.

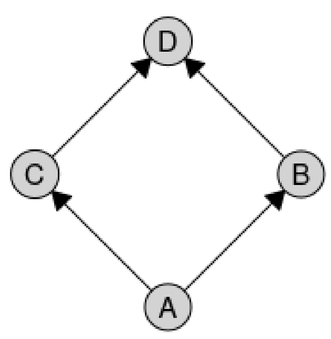
Key Contributions

- **High-performance backend:** Rust implementation with efficient graph traversal.
- **Type-safe graph classes:** Enforce DAG, PDAG, ADMG, UG constraints at the graph level.
- **Expressive syntax:** Compose graphs with concise R formulas.
- **Scalable queries:** Efficient evaluation of ancestry and neighborhood relations in large graphs.

Basic Usage

Syntax like a picture in your head:

```
cg <- caugi(
  A %-->% B + C,
  B %-->% D,
  C %-->% D,
  class = "DAG"
)
plot(cg)
```



Querying and Metrics

- **Relational queries**: `parents()`, `ancestors()`, `neighbors()`, etc.
- **Structural queries**: `is_acyclic()`, `is_cpdag()`, etc.
- **Causal queries**: `adjustment_set()`, `d_separated()`, etc.
- **Graph metrics**: `shd()`, `aid()`, etc.

Example queries:

```
> parents(cg, "D")
[1] "B" "C"

> d_separated(cg, "A", "D", Z = c("B", "D"))
[1] TRUE
```

How it Works

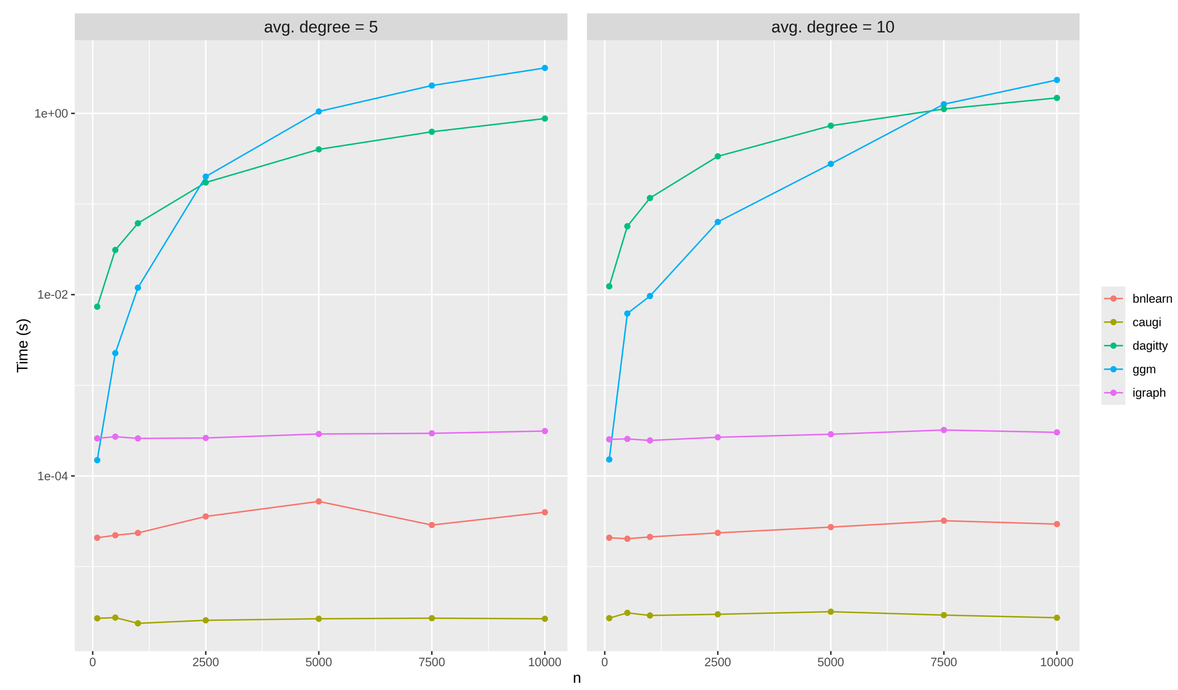
- **Backend:** Rust for memory safety and performance.
- **Storage:** Compressed Sparse Row (CSR)
  - Memory-efficient for sparse graphs
  - Direct slice access ⟶ O(1) adjacency lookup
- **Immutable + lazy rebuild:**
  - Rebuilds in O(|V| + |E|) on query after graph modification.
- **Result:** Fast, predictable queries with consistent graph state.

Graph-class Safe

- Think type safety, but for graphs.
- Supports DAGs, PDAGs, ADMGs, UGs, and unknown graphs.
- All graph modifications are checked against class constraints.
  - E.g., adding an edge that creates a cycle in a DAG is prevented.
- Ensures reliable, consistent graph state throughout analysis.

Benchmarks

- Benchmarked `caugi` against `bnlearn`, `dagitty`, `ggm`, and `igraph` for parent and child queries on DAGs[1].
  - We report median runtime from bench package.
- `caugi` consistently achieved the lowest computation times across graph sizes and degree settings.
- Additional queries (ancestors, descendants, d-separation) showed the same ranking, with `caugi` often around an order of magnitude faster.



Contact

- Pkgdown site: caugi.org
- GitHub: frederikfabriciusbjerre/caugi

[1]Benchmark environment: R 4.5.3, x86_64-pc-linux-gnu, Linux Mint 22.3, CPU: AMD Ryzen 7 8845HS, RAM: 14Gi. Using bench package for benchmarking.